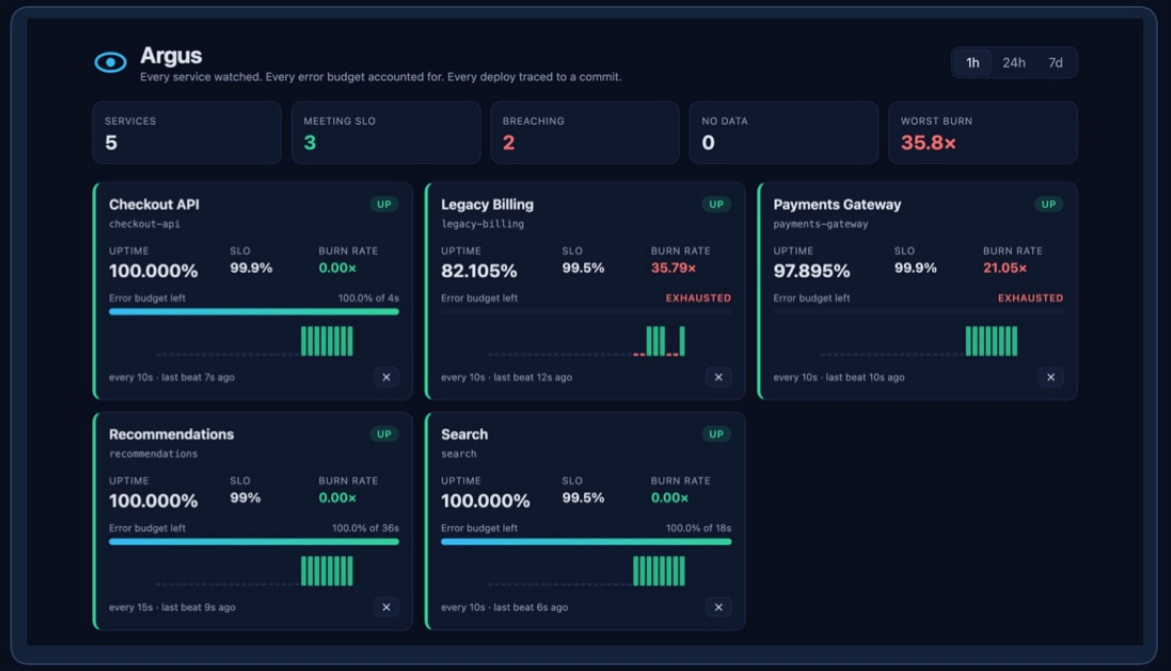
Argus

SLO and uptime monitoring, shipped through a complete DevOps pipeline

Every service watched. Every error budget accounted for.
Every deploy traced to a commit.



The live status board, served through the Kubernetes Ingress

Arnav Yadav
Session 21 DevOps capstone
github.com/arnav-yadav/argus

THE APPLICATION

Liveness is inferred from silence

1

Register

A service registers with a heartbeat interval and an SLO target. Argus returns a one-time ingest token and stores only its SHA-256 hash.

2

Beat

Every interval it reports ok, degraded or fail, with the token in the X-Argus-Token header. A wrong token gets a 401.

3

Silence = down

Miss two intervals and the service is DOWN, though it never said so. A dead process can't report its own death.

4

Compute SLOs

Uptime, error budget and burn rate over 1h, 24h and 7d, served to the board, the REST API and Prometheus.

Built with FastAPI · PostgreSQL + Alembic · React + nginx · a heartbeat simulator with five reliability profiles

THE SLO MATHS

Error budgets measured in seconds

Time is cut into slots, one heartbeat interval each. A slot is good if an ok or degraded beat landed in it.



9 good of 11 judged · the dashed slot is still in progress

- Slots, not beat counts: a chatty service can't inflate its uptime
- Windows are clipped to the service's creation time
- The in-progress slot counts only once it has a beat

86.4 s — the daily error budget at a 99.9% SLO: (1 − 0.999) × 86,400 s

1× — burn rate that spends exactly the budget over the window

14.4× — pages someone: a 30-day budget would be gone in about two days

THE DELIVERY CHAIN

From commit to cluster in 13 stages

1. **Application** — FastAPI + React
2. **Git commit** — one change each
3. **CI pipeline** — GitHub Actions
4. **Docker image** — multi-stage, non-root
5. **Security scan** — Trivy, amd64 + arm64
6. **Registry** — GHCR, SHA tags
7. **Terraform** — AWS VPC + EKS
8. **Kubernetes** — minikube and EKS
9. **Helm** — one chart, 4 values files
10. **Ingress** — / and /api
11. **Autoscaling** — HPA, 2 to 6
12. **Monitoring** — Prometheus, Grafana
13. **Troubleshooting** — six real incidents

The order the course asks the final demo to walk through. The rest of the talk follows it.

48 tests, 98% coverage, no live database

13

routes: 9 under /api, plus /, /health, /ready and /metrics

2

tables, created by an Alembic migration

48

tests across the API, migrations and SLO maths

98%

line coverage, with ruff and bandit rules clean

How it's tested

- The SLO maths are pure functions that take now as a parameter, so they are tested without mocking the clock
- Tests run against an in-memory SQLite database set up in conftest.py, never the real one
- Migrations are tested up and down, and run under a Postgres advisory lock, so six replicas starting at once still migrate exactly once
- CI runs ruff (with bandit security rules) and pytest before any image is built

Every push to main ships in under 4 minutes



Runs on every push to main

Tags are commit SHAs, never latest

GitHub never holds cluster credentials

04 · DOCKER IMAGE

Small images that run as non-root

BEFORE · Debian slim

306 MB

44 HIGH CVEs Debian hasn't fixed, in util-linux, perl, ncurses and systemd

→

AFTER · Alpine

178 MB

0 vulnerabilities at any severity, in amd64 and arm64

- ✓ Multi-stage builds: no compilers at runtime
- ✓ No pip in the Python runtime images
- ✓ Health checks in every image
- ✓ Non-root users: 10001 backend, 101 frontend, 10002 simulator
- ✓ Read-only root filesystem, all capabilities dropped
- ✓ Compose: Postgres → one-shot migrate → backend → frontend

The Trivy gate earned its place

1

Run 1: blocked

Trivy stopped the backend and simulator: 4 HIGH CVEs in the urllib3, msgpack and setuptools code that pip bundles inside itself.

2

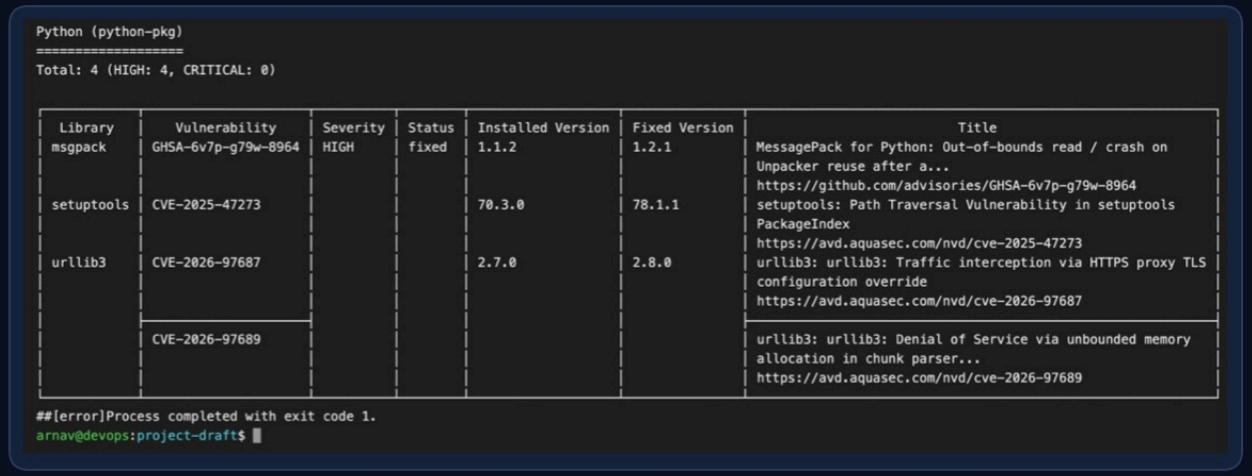
Fix, not suppress

Nothing installs packages at runtime, so pip and ensurepip were removed from the images. The next run passed.

3

Tightened

Backend moved to Alpine, both architectures are scanned, and the gate now fails on any HIGH or CRITICAL, fixed upstream or not.



```
Python (python-pkg)
==================
Total: 4 (HIGH: 4, CRITICAL: 0)

┌──────────┬──────────────────┬──────────┬────────┬───────────────────┬───────────────┬──────────────────────────────────────────────────────────────┐
│ Library  │  Vulnerability   │ Severity │ Status │ Installed Version │ Fixed Version │                            Title                             │
├──────────┼──────────────────┼──────────┼────────┼───────────────────┼───────────────┼──────────────────────────────────────────────────────────────┤
│ msgpack  │ GHSA-6v7p-g79w-8964 │ HIGH  │ fixed  │ 1.1.2             │ 1.2.1         │ MessagePack for Python: Out-of-bounds read / crash on        │
│          │                  │          │        │                   │               │ Unpacker reuse after a...                                    │
│          │                  │          │        │                   │               │ https://github.com/advisories/GHSA-6v7p-g79w-8964            │
│ setuptools │ CVE-2025-47273 │          │        │ 70.3.0            │ 78.1.1        │ setuptools: Path Traversal Vulnerability in setuptools       │
│          │                  │          │        │                   │               │ PackageIndex                                                 │
│          │                  │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2025-47273                   │
│ urllib3  │ CVE-2026-97687   │          │        │ 2.7.0             │ 2.8.0         │ urllib3: urllib3: Traffic interception via HTTPS proxy TLS   │
│          │                  │          │        │                   │               │ configuration override                                       │
│          │                  │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-97687                   │
│          ├──────────────────┤          │        │                   │               ├──────────────────────────────────────────────────────────────┤
│          │ CVE-2026-97689   │          │        │                   │               │ urllib3: urllib3: Denial of Service via unbounded memory     │
│          │                  │          │        │                   │               │ allocation in chunk parser...                                │
│          │                  │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-97689                   │
└──────────┴──────────────────┴──────────┴────────┴───────────────────┴───────────────┴──────────────────────────────────────────────────────────────┘
##[error]Process completed with exit code 1.
arnav@devops:project-draft$
```

Run 1, real gate output: four HIGH findings, exit code 1, nothing pushed

Registry: GitHub Container Registry

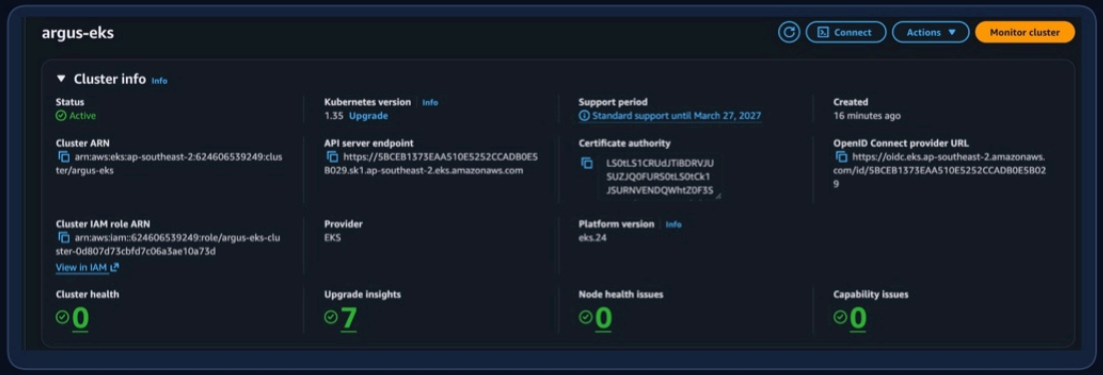
`sha-1a2b3c4` and the full SHA, never latest. Multi-arch, with an SBOM attached. Results go to GitHub code scanning: 0 open alerts.

AWS VPC and EKS, built and torn down

52
AWS resources

8 min
for the EKS control plane

0
left after destroy, checked in AWS



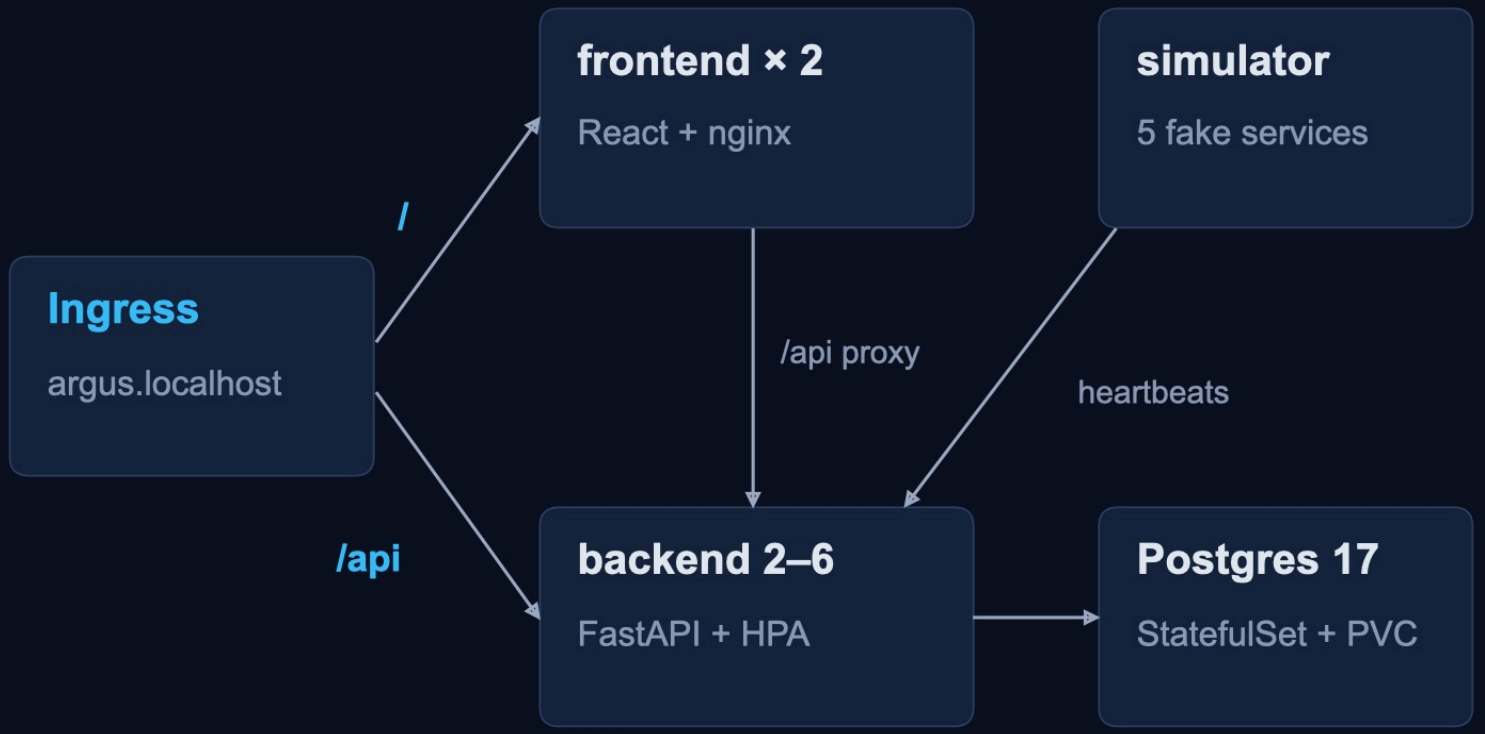
In the AWS console: Active, Kubernetes 1.35

Hardened along HashiCorp's and AWS's guidance

- Node disks encrypted (the account's default is off)
- Kubernetes API reachable only from one IP
- Access entries only, no aws-auth ConfigMap
- Audit logs in CloudWatch · node auto repair · IMDSv2

- VPC: 2 public + 2 private subnets, one NAT gateway
- EKS 1.35, pinned to standard support
- 2 × m7i-flex.large in ap-southeast-2
- Applied three times, destroyed each time: under $1

One chart, two environments



main namespace argus

Deployed by ArgoCD from main, with values-minikube.yaml. Has the ServiceMonitor, alert rules and dashboard.

staging namespace argus-staging

Installed with helm upgrade --install, checked with helm test, served at staging.argus.localhost.

Pod Security “restricted” · non-root · read-only root filesystem · migrations in an init container

Under load, 2 replicas became 6

2 → 6

backend replicas, 30 seconds after CPU crossed the target

- HorizontalPodAutoscaler on the backend
- Load: 12 clients for 3 minutes, through the Ingress
- Scales back to 2 after a 5-minute cooldown

HPA scales the backend under load

```
arnav@devops:project-draft$ kubectl --context argus -n argus get hpa argus-backend
NAME            REFERENCE                   TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
argus-backend   Deployment/argus-backend   cpu: 9%/70%   2         6         2          3h36m
arnav@devops:project-draft$ seq 12 | xargs -P12 -I{} bash -c 'end=$((SECONDS+180)); while [ $SECONDS -lt $end ]; do curl -s -o /dev/null "http://argus.localhost:8080/api/slo/bu
e' > /dev/null 2>&1 & echo "12 parallel clients hammering /api/slo/budget for 3 minutes (pid $!)"
12 parallel clients hammering /api/slo/budget for 3 minutes (pid 37659)
arnav@devops:project-draft$ for i in $(seq 1 11); do sleep 15; echo "$(date +%H:%M:%S)  $(kubectl --context argus -n argus get hpa argus-backend --no-headers | awk '{print $3,
"; done
22:51:27  cpu: 8%/70% replicas=2
22:51:42  cpu: 8%/70% replicas=2
22:51:57  cpu: 8%/70% replicas=2
22:52:12  cpu: 8%/70% replicas=2
22:52:27  cpu: 373%/70% replicas=2
22:52:42  cpu: 373%/70% replicas=4
22:52:57  cpu: 373%/70% replicas=6
22:53:13  cpu: 373%/70% replicas=6
22:53:28  cpu: 383%/70% replicas=6
22:53:43  cpu: 383%/70% replicas=6
22:53:58  cpu: 383%/70% replicas=6
arnav@devops:project-draft$
```

Real output: kubectl get hpa every 15 seconds while the load ran

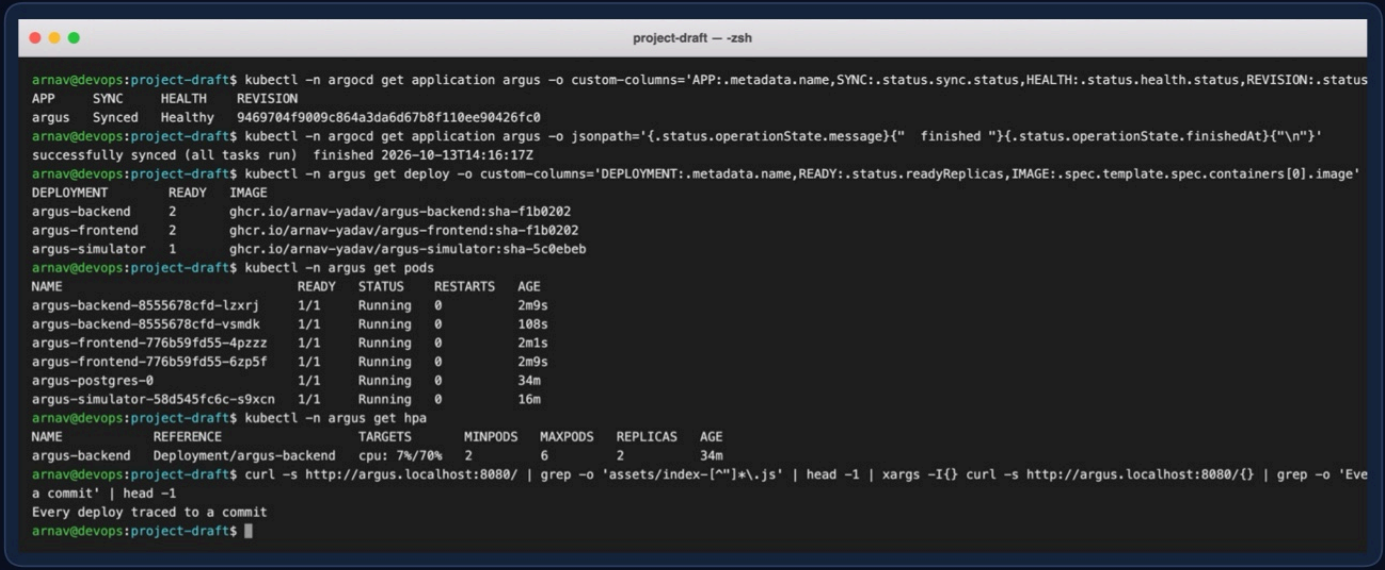
70%
CPU target

383%
of the target at peak

0
heartbeats missed

A commit deploys itself

1. **Commit** — change the tagline and push
2. **Pipeline** — tests, images, Trivy, GHCR: about 3 min
3. **Promote** — a bot commits the new tag [skip ci]
4. **ArgoCD** — notices within a minute, then syncs
5. **Live** — pods roll; the board shows the new SHA



Recorded run

- sha-f1b0202 went from commit to pods with no kubectl or helm by hand
- ArgoCD: Synced and Healthy
- Healthy services stayed at 100% through the rollout

Metrics, burn-rate alerts, a dashboard



Provisioned automatically from a ConfigMap: per-service uptime, budgets and burn rates

Alert	Fires when	Severity
Fast burn	1h & 5m > 14.4×	**page**
Medium burn	6h & 30m > 6×	**page**
Slow burn	24h & 1h > 1×	**ticket**
Service down	down for 1 min	**page**
Budget exhausted	24h budget < 0	**ticket**

SLO gauges are computed from the database at scrape time, so every replica reports the same numbers. honorLabels keeps each monitored service's label intact.

Six real incidents, six root causes

Trivy blocked the first push

Cause pip vendors old urllib3, msgpack and setuptools

Fix remove pip from the runtime images

All services merged into one Prometheus series

Cause our service label clashed with Kubernetes' own

Fix honorLabels on the ServiceMonitor

Grafana showed “No data”

Cause the pod was OOMKilled at 384 Mi

Fix raise the memory limit to 768 Mi

Every deploy looked like an outage

Cause two simulators kept rotating each other's tokens

Fix Recreate strategy, pinned simulator tag

The first terraform apply stopped

Cause the AWS org policy denies OIDC providers

Fix enable_irsa = false, Pod Identity instead

A new service showed −5782% budget

Cause budget was extrapolated from a ratio

Fix measure the budget in bad seconds

RESULTS

Every rubric module, evidenced

48
tests, 98% coverage

9
CI jobs on every push

0
vulnerabilities in 6 image builds

52
AWS resources from Terraform

5
Prometheus alert rules

2 → 6
replicas under load

0
missed heartbeats while scaling

38
screenshots of real output

All the evidence, module by module: `docs/EVIDENCE.md` in the repository



Thank you

Live demo: commit → pipeline → GitOps rollout

`github.com/arnav-yadav/argus`

Arnav Yadav · Session 21 DevOps capstone

Questions?